

SECURITY VULNERABILITY REPORT

Static Analysis & Threat Assessment

Repository: [OmarRao/analyzer](#)

Branch: main · Analysis Engine: Semgrep + MITRE ATT&CK Mapping

100

/ 100

CRITICAL

Composite Risk Score

0

20

45

70

100

LOW

MEDIUM

HIGH

CRITICAL

SCORING BREAKDOWN

Critical findings × 10

+140

Warning findings × 3

+69

Dependency CVEs × 8

+0

Capped at

100

14

Critical Findings

23

Warnings

0

Dependency CVEs

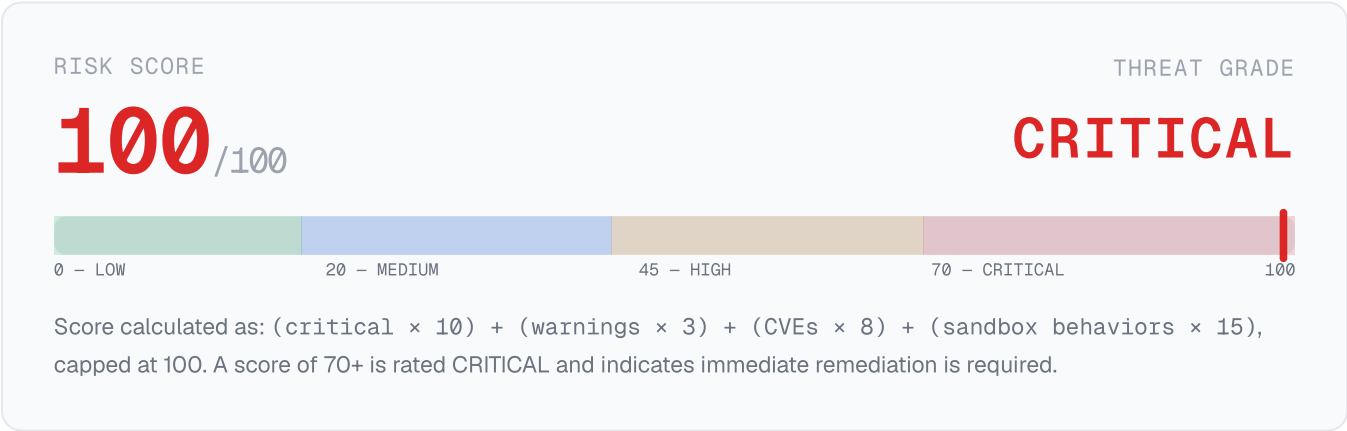
10

CWE Categories

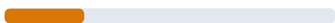
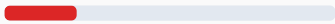
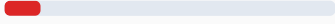
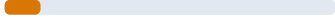
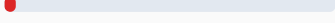
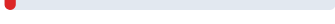
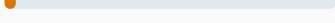
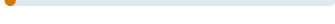
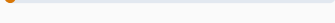
Scan Overview

REPOSITORY		FINDING BREAKDOWN	
Owner	OmarRao	Total Findings	37
Repository	analyzer	Critical (ERROR)	14
Branch	main	Warning	23
Analysis Engine	Semgrep	Dependency CVEs	0
Generated	June 10, 2026 at 11:35 UTC	CWE Categories Hit	10

Composite Risk Assessment



Vulnerability Category Breakdown

CWE IDENTIFIER	COUNT	SHARE	DISTRIBUTION
CWE-79	9	24%	
CWE-704	8	22%	
CWE-89	7	19%	
CWE-78	4	11%	
CWE-327	4	11%	
CWE-250	1	3%	
CWE-918	1	3%	
CWE-22	1	3%	
CWE-668	1	3%	
CWE-489	1	3%	

Attack Surface Analysis

Each vulnerability class is mapped to the corresponding MITRE ATT&CK technique and tactic. Counts reflect findings from the static analysis phase. Exposed vectors require immediate remediation per the AI Fix Advisor output.

Vulnerability Type	CWE	ATT&CK Technique	Status
SQL Injection	CWE-89	T1190	7 findings
Command Injection	CWE-78	T1059	4 findings
Cross-Site Scripting	CWE-79	T1059.007	9 findings
SSRF	CWE-918	T1090	1 finding
Path Traversal	CWE-22	T1083	1 finding
Hardcoded Credentials	CWE-798	T1552.001	Clear
Weak Cryptography	CWE-327	T1600	4 findings
Insecure Deserialization	CWE-502	T1059	Clear

TOP PRIORITY FINDINGS

Critical Issues Requiring Immediate Action

Missing User

CRITICAL

Dockerfile · line 12

By not specifying a USER, a program in the container may run as 'root'. This is a security hazard. If an attacker can control a process running as root, they may have control over the container. Ensure that the last USER

CWE-250

Tainted Sql String

CRITICAL

app.py · line 72

Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction could accidentally result in a SQL injection. An attacker could use a SQL injection to steal or modify

CWE-704

Tainted Sql String

CRITICAL

app.py · line 111

Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction could accidentally result in a SQL injection. An attacker could use a SQL injection to steal or modify

CWE-704

Subprocess Injection

CRITICAL

app.py · line 125

Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerability. An attacker could use this vulnerability to execute arbitrary commands on the host, which allows them to

CWE-78

T1059

Execution

Dangerous Subprocess Use

CRITICAL

app.py · line 125

Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform command injection. You may consider using 'shlex.escape()'.

CWE-78

T1059

Execution

Subprocess Shell True

CRITICAL

app.py · line 125

Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command using a shell process. Doing so propagates current shell settings and variables, which makes it muc

CWE-78

T1059

Execution

Dangerous System Call

CRITICAL

app.py · line 145

Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use the 'subprocess' module instead, which is easier to use without accidentally exposing a command injection vuln

CWE-78

T1059

Execution

Tainted Sql String

CRITICAL

app.py · line 155

Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction could accidentally result in a SQL injection. An attacker could use a SQL injection to steal or modify

CWE-704

Complete Findings Table (37 findings)

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	Missing User	Dockerfile:12	CWE-250		By not specifying a USER, a program in the container may run as 'root'. This is a security hazard. If an attacker can co
CRITICAL	Tainted Sql String	app.py:72	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	Tainted Sql String	app.py:111	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	Subprocess Injection	app.py:125	CWE-78	T1059	Detected user input entering a 'subprocess' call unsafely. This could result in a command injection vulnerability. An at

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	Dangerous Subprocess Use	app.py:125	CWE-78	T1059	Detected subprocess function 'check_output' with user controlled data. A malicious actor could leverage this to perform
CRITICAL	Subprocess Shell True	app.py:125	CWE-78	T1059	Found 'subprocess' function 'check_output' with 'shell=True'. This is dangerous because this call will spawn the command
CRITICAL	Dangerous System Call	app.py:145	CWE-78	T1059	Found user-controlled data used in a system call. This could allow a malicious actor to execute commands. Use the 'subpr
CRITICAL	Tainted Sql String	app.py:155	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	Tainted Sql String	app.py:175	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
CRITICAL	Tainted Sql String	app.py:177	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	Tainted Sql String	app.py:178	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	Tainted Sql String	app.py:179	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	Tainted Sql String	app.py:206	CWE-704		Detected user input used to manually construct a SQL string. This is usually bad practice because manual construction co
CRITICAL	Ssrf Injection Urllib	app.py:218	CWE-918	T1090	Data from request object is passed to a new server-side request. This could lead to a server-side request forgery (SSRF)

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	Sql Injection Db Cursor Execute	app.py:67	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	Sql Injection Db Cursor Execute	app.py:68	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	Sql Injection Db Cursor Execute	app.py:105	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	Directly Returned Format String	app.py:116	CWE-79	T1059.007	Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user input can re
WARNING	Raw Html Format	app.py:116	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	Raw Html Format	app.py:116	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	Directly Returned Format String	app.py:126	CWE-79	T1059.007	Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user input can re
WARNING	Raw Html Format	app.py:126	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	Raw Html Format	app.py:126	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	Path Traversal Open	app.py:132	CWE-22	T1083	Found request data in a call to 'open'. Ensure the request data is validated or sanitized, otherwise it could result in

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	Directly Returned Format String	app.py:136	CWE-79	T1059.007	Detected Flask route directly returning a formatted string. This is subject to cross-site scripting if user input can re
WARNING	Raw Html Format	app.py:136	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	Raw Html Format	app.py:136	CWE-79	T1059.007	Detected user input flowing into a manually constructed HTML string. You may be accidentally bypassing secure methods of
WARNING	Sql Injection Db Cursor Execute	app.py:170	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	Sql Injection Db Cursor Execute	app.py:170	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	Sql Injection Db Cursor Execute	app.py:171	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	Sql Injection Db Cursor Execute	app.py:199	CWE-89	T1190	User-controlled data from a request is passed to 'execute()'. This could lead to a SQL injection and therefore protected
WARNING	Insecure Hash Algorithm Md5	app.py:203	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a
WARNING	Avoid_App_Run_With_Bad_Host	app.py:226	CWE-668		Running flask app with host 0.0.0.0 could expose the server publicly.
WARNING	Debug Enabled	app.py:226	CWE-489		Detected Flask app with debug=True. Do not deploy to production with this flag enabled as it will leak sensitive informa

SEVERITY	RULE / VULNERABILITY	FILE & LINE	CWE	ATT&CK	DESCRIPTION
WARNING	Insecure Hash Algorithm Md5	utils.py:13	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a
WARNING	Md5 Used As Password	utils.py:13	CWE-327	T1600	It looks like MD5 is used as a password hash. MD5 is not considered a secure password hash because it can be cracked by
WARNING	Insecure Hash Algorithm Md5	utils.py:43	CWE-327	T1600	Detected MD5 hash algorithm which is considered insecure. MD5 is not collision resistant and is therefore not suitable a

How This Report Was Generated

ANALYSIS ENGINE		SCORING MODEL	
Static Analysis	Semgrep OSS	Critical weight	× 10 per finding
Rulesets	OWASP, CWE-25, Secrets	Warning weight	× 3 per finding
Threat Framework	MITRE ATT&CK v14	CVE weight	× 8 per CVE
CVE Audit	pip-audit / npm audit	Score cap	100

RESPONSIBLE DISCLOSURE

This report is generated automatically by SecureScope and is intended for the repository owner and their authorised security team. Findings reflect the state of the codebase at the time of analysis. They should be validated by a qualified security engineer before remediation work begins.

SecureScope maps findings to MITRE ATT&CK v14 and CWE identifiers. These mappings are best-effort and should not be treated as a definitive classification. False positives are possible — Semgrep static analysis is pattern-based and does not perform dynamic execution.

For questions about this report or the SecureScope platform, refer to the project repository at github.com/OmarRao/secure-scope.

SecureScope^{BETA}

AI-Powered GitHub Security Analysis · MITRE ATT&CK v14 · CWE Top 25

Omar Rao

Engineer by trait
Data Resilience, Privacy & Cybersecurity
Expert
Founder, SecureScope